PROJECT REPORT

HematoVision : Advanced Blood Cell Classification Using Transfer Learning

1.INTRODUCTION

1.1 Project Overview:

HematoVision: Advanced Blood Cell Classification Using Transfer Learning is an AI-based web application designed to automatically identify and classify different types of blood cells from microscopic images. The project uses transfer learning with a pre-trained deep learning model to improve classification accuracy and reduce training time.

1.2 Purpose:

The purpose of the HematoVision project is to develop an automated system for accurate and efficient classification of blood cells using deep learning and transfer learning techniques. Manual identification of blood cells is time-consuming and requires expert knowledge, which may lead to delays and human errors.

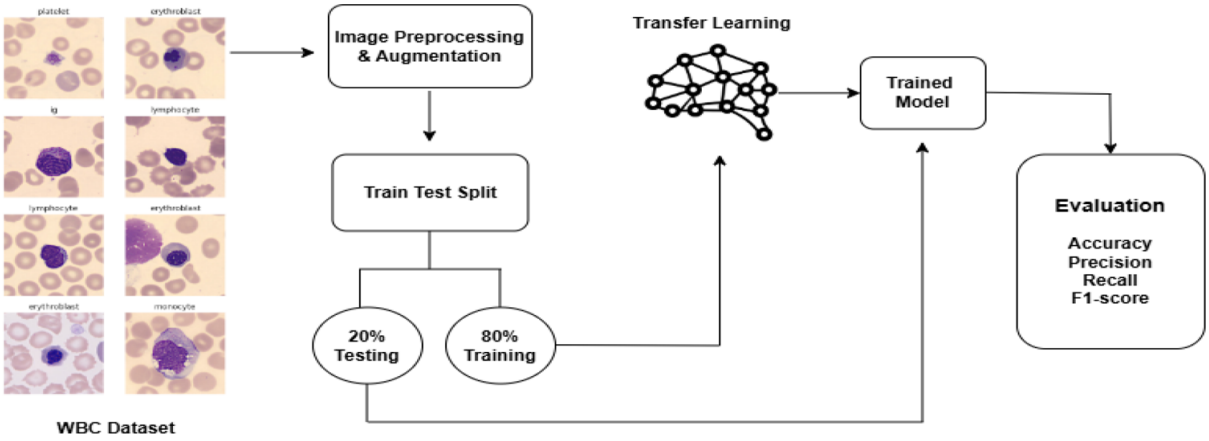
2. IDEATION PHASE

2.1 Problem Statement:

Accurate classification of blood cells is essential for diagnosing various diseases such as infections, anemia, leukemia, and immune disorders. Traditionally, this process is performed manually by medical experts using microscopic examination, which is time-consuming, labor-intensive, and prone to human error. The increasing number of medical cases creates a need for faster and more reliable diagnostic methods.

2.2 Empathy Map Canvas:

Medical professionals need a fast and accurate system to classify blood cells without relying entirely on manual observation. They face challenges such as time constraints, heavy workload, and risk of human error during diagnosis. They expect a reliable, easy-to-use tool that provides quick and precise results. HematoVision helps them by automating blood cell classification, saving time and improving diagnostic efficiency.



2.3 Brainstorming:

Brainstorming was conducted to identify an effective solution for automatic blood cell classification using artificial intelligence. Various ideas such as manual image analysis, traditional machine learning, and deep learning approaches were discussed. Transfer learning was selected as the best approach due to its high accuracy and reduced training time. The idea of developing a web-based application using Flask was finalized to make the system user-friendly and accessible.

3. REQUIREMENT ANALYSIS

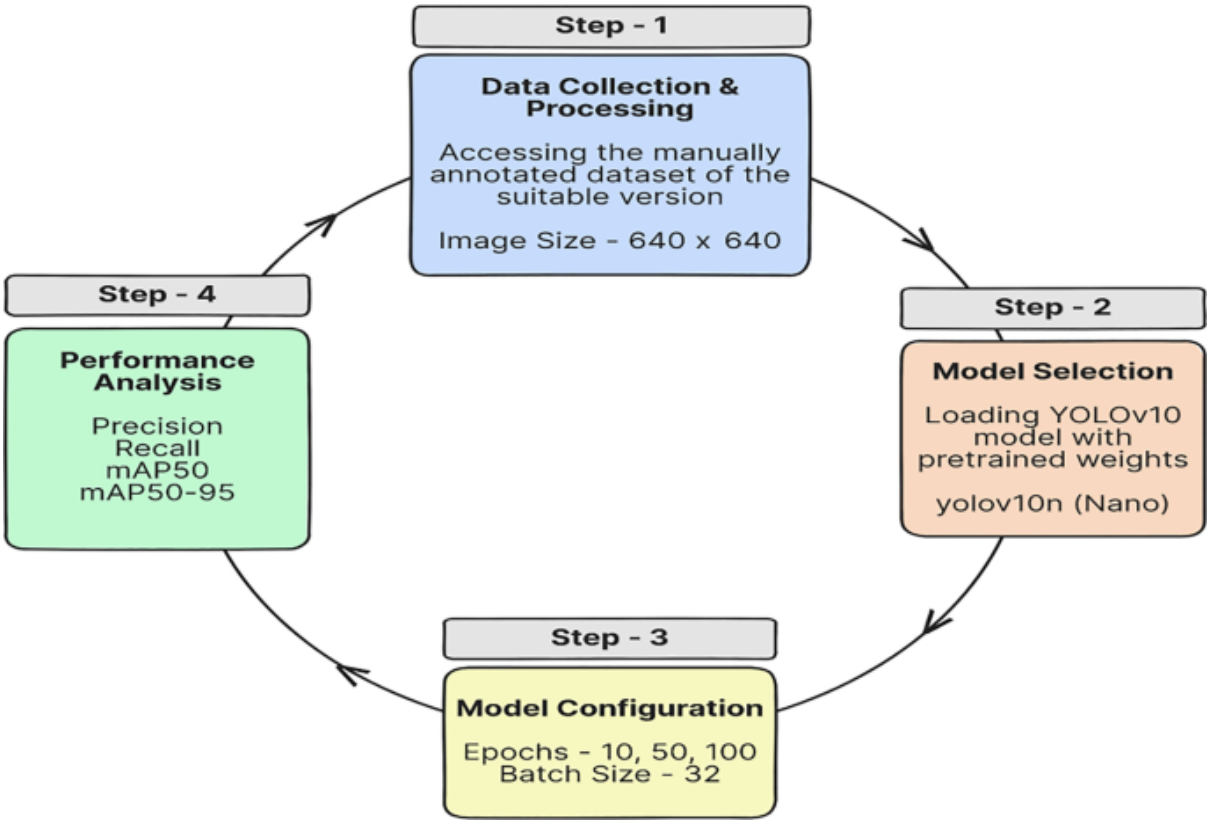
3.1 Customer Journey map:

The user visits the HematoVision web application through a browser and accesses the home page. They upload a microscopic blood cell image using the upload option provided. The system processes the image using the trained deep learning model and predicts the blood cell type. The result is displayed clearly on the screen. The user can then upload another image if needed, making the system simple, fast, and efficient to use.

3.2 Solution Requirement:

The HematoVision system requires both hardware and software components to ensure efficient blood cell classification. The system needs a computer with sufficient processing power, minimum 8 GB RAM, and internet access to run the web application smoothly. On the software side, Python programming language, Flask framework, TensorFlow/Keras for deep learning, and HTML/CSS for the user interface are required.

3.3 Data Flow Diagram:



3.4 Technology Stack:

The HematoVision project uses a combination of machine learning and web development technologies to build an efficient blood cell classification system. Python is used as the main programming language for model development and backend processing. TensorFlow and Keras are used to implement transfer learning and build the deep learning classification model.

Flask is used as the web framework to connect the frontend and backend. HTML and CSS are used to design the user interface for image upload and result display. NumPy is used for image preprocessing and numerical operations. The system is developed and tested using tools such as VS Code and runs on a web browser for easy access.

4. PROJECT DESIGN

4.1 Problem Solution Fit:

The HematoVision project effectively addresses the problem of manual blood cell classification, which is time-consuming, requires expert knowledge, and

is prone to human error. Medical professionals need a faster and more accurate method to identify blood cell types from microscopic images.

4.2 Proposed Solution:

The proposed solution is HematoVision, an AI-powered web application that automatically classifies blood cells using transfer learning and deep learning techniques. The system uses a pre-trained convolutional neural network (CNN) model that has been trained on blood cell image datasets to accurately identify different types of blood cells such as Basophils, Eosinophils, Lymphocytes, Monocytes, and Neutrophils.

4.3 Solution Architecture:

The HematoVision system follows a client–server architecture integrating a web interface, backend server, and deep learning model. The user interacts with the frontend web application developed using HTML and CSS to upload blood cell images. The Flask backend server receives the image and performs preprocessing such as resizing and normalization.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning:

The project was planned in phases including problem identification, dataset collection, model development, and web application integration. Transfer learning was selected to improve accuracy and reduce training time. Flask was chosen for backend development and HTML/CSS for frontend design. Proper testing and evaluation were conducted before final deployment.

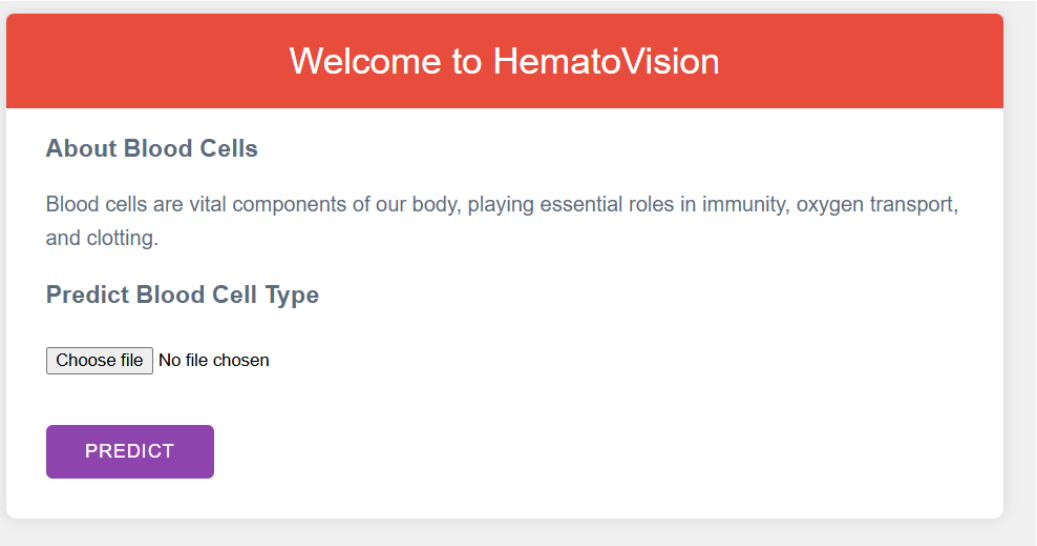
6. FUNCTIONAL AND PERFORMANCE TESTING

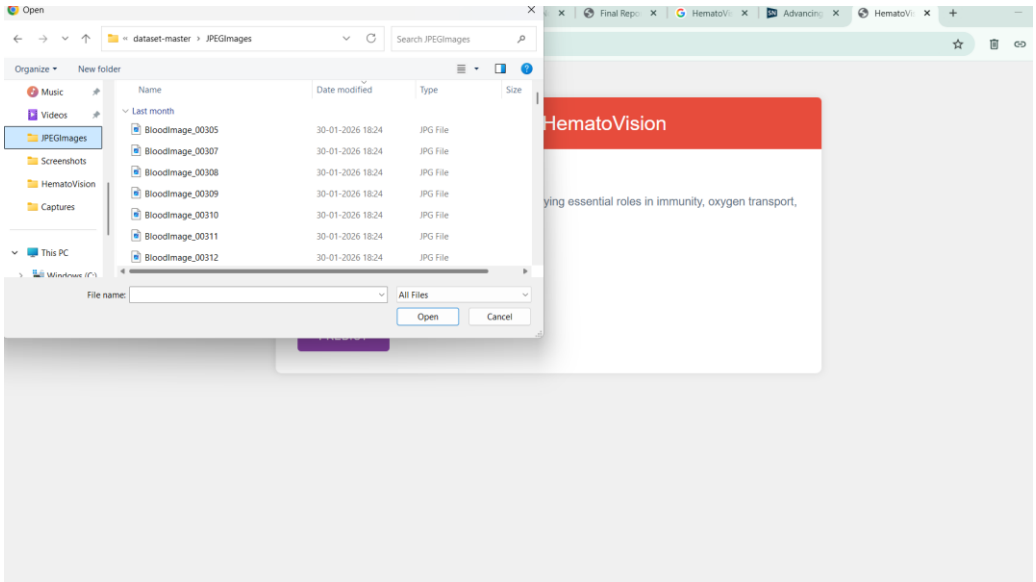
6.1 Performance Testing:

The system was tested using multiple blood cell images to evaluate prediction accuracy and response time. The model showed high accuracy in classifying different blood cell types. The web application responded quickly to user uploads and displayed results efficiently. Performance testing confirmed the system is reliable, fast, and suitable for real-time use.

7. RESULTS

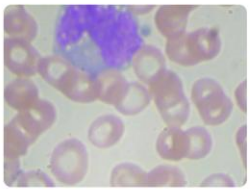
7.1 Output Screenshots:





127.0.0.1:5000/predict

Prediction Result



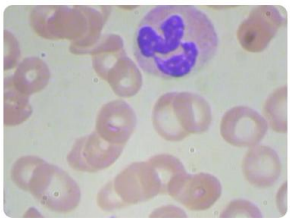
Predicted Blood Cell Type:

Lymphocyte

Upload Another Image

127.0.0.1:5000/predict

Prediction Result

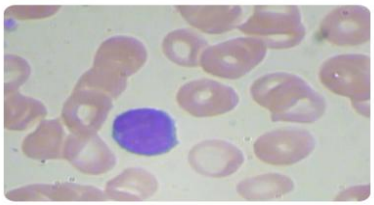


Predicted Blood Cell Type:

Monocyte

Upload Another Image

Prediction Result



Predicted Blood Cell Type:

Basophil

Upload Another Image

8. ADVANTAGES & DISADVANTAGES

Advantages:

The system provides fast and accurate blood cell classification using AI technology. It reduces manual effort and minimizes human errors in diagnosis. The web interface is user-friendly and easy to use. It demonstrates the real-world application of deep learning in healthcare.

Disadvantages:

The system accuracy depends on the quality and size of the training dataset. It requires a trained model and proper system setup to function correctly. The system cannot replace medical experts completely. It needs further improvement for large-scale clinical use.

9. CONCLUSION

HematoVision successfully demonstrates automatic blood cell classification using transfer learning. The system provides accurate, fast, and reliable predictions through a web-based interface. It reduces manual effort and improves efficiency in identifying blood cells. This project shows the potential of AI in supporting healthcare and medical diagnosis.

10. FUTURE SCOPE

The system can be improved by using larger and more diverse datasets to increase accuracy. Mobile application integration can make the system more accessible. Additional features such as disease detection can be added. The system can be enhanced for real-time clinical and hospital use.

11. APPENDIX

Source Code (if any):

HEMATOVISION
- dataset
- models
- static
- templates
  - home.html
  - result.html
- uploads
- venv
- app.py 2
- requirements.txt
- train.py 4

OUTLINE

templates > home.html > html

```html
<!DOCTYPE html>
<html>
<head>
    <title>HematoVision</title>

    <style>

        body {
            background-color: #f0f0f0;
            font-family: Arial, sans-serif;
        }

        .container {
            width: 700px;
            margin: 50px auto;
            background: #ffffff;
            padding: 30px;
            border-radius: 8px;
            box-shadow: 0px 0px 10px rgba(0,0,0,0.1);
        }

        .header {
            background-color: #e74c3c;
            color: white;
            text-align: center;
            padding: 20px;
            font-size: 28px;
            border-radius: 6px 6px 0 0;
            margin: -30px -30px 20px -30px;
        }

        h3 {
            color: #5a6c7d;
            margin-top: 20px;
        }
```

app.py 2 | requirements.txt | train.py 4 | home.html | result.html

templates > result.html > html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Prediction Result</title>

    <style>
        body {
            text-align: center;
            font-family: Arial;
            background: #f4f6f8;
            margin-top: 50px;
        }

        .box {
            background: white;
            padding: 30px;
            display: inline-block;
            border-radius: 10px;
            box-shadow: 0px 0px 10px gray;
        }

        img {
            border-radius: 10px;
        }

        button {
            padding: 10px 20px;
            background: #e74c3c;
            color: white;
            border: none;
            border-radius: 5px;
            margin-top: 20px;
        }

        button:hover {
            background: #c0392b;
        }
```

Ln 61, Col 8 Spaces: 4 UTF-8 LF

app.py 2 | requirements.txt | train.py 4 | home.html | result.html

app.py > ...

```python
import os
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'

from flask import Flask, render_template, request, redirect, url_for
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image
import numpy as np

app = Flask(__name__)

# Upload folder
UPLOAD_FOLDER = "static/uploads"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

# Load model
print("Loading HematoVision model...")
model = load_model("models/hematovision_model.h5")
print("Model loaded successfully!")

# Class names
class_names = [
    "Basophil",
    "Eosinophil",
    "Lymphocyte",
    "Monocyte",
    "Neutrophil"
]

# Home page
@app.route("/")
def home():
    return render_template("home.html")

# Predict route
@app.route("/predict", methods=["POST"])
def predict():
```

requirements.txt

```
Flask==2.3.3
tensorflow==2.20.0
numpy
Pillow
opencv-python
Werkzeug==2.3.7
```

Dataset Link:

The dataset used contains microscopic images of different blood cell types. It was used for training and testing the transfer learning model. Publicly available medical image datasets were used for model training. Example source: Kaggle Blood Cell Images Dataset.

https://www.kaggle.com/datasets/paultimothymooney/blood-cells

GitHub Link:

https://github.com/Kotamprakash/HematoVision-Advanced-Blood-Cell-Classification-Using-Transfer-Learning